检索增强生成与上下文工程

RAG & Context Engineering

原文作者：@techNmak。

先列专业术语对照表，再提供逐页完整中文译文。每个原文页面的译文之后附原文矢量对照页，完整保留公式、图表、代码和英文引用，便于核对。

正文中文：LXGW WenKai（霞鹜文楷）；英文与数字：Times New Roman。原文对照页保留原始字体。

本文采用翻译辅助服务生成初稿，统一技术术语并检查结构、字体和导航；涉及精确数值、公式或实现细节时，可直接对照相邻原文页。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Retrieval-augmented Generation	检索增强生成	RAG
Hierarchical Retrieval	层次检索	
Multi-query Retrieval	多查询检索	
Continued Pretraining	继续预训练	
Parametric Knowledge	参数化知识	
Contextual Retrieval	上下文检索	
Query Transformation	查询变换	
Context Engineering	上下文工程	
Context Compression	上下文压缩	
Prompt Engineering	提示词工程	
Semantic Chunking	语义分块	
Context Assembly	上下文组装	
Context Window	上下文窗口	
Chunk Overlap	块重叠	
Late Chunking	延迟分块	
Ground Truth	基准真值	
Rank Fusion	排名融合	
Fine-tuning	微调	
Re-indexing	重新建立索引	
Performance	性能	
Chunk Size	块大小	
Provenance	来源溯源	
Generation	生成	
Retrieval	检索	
Retriever	检索器	
Reranking	重排序	
Citation	引用	
Evidence	证据	
Pipeline	流水线	
Parsing	解析	
Latency	延迟	
Query	查询	Q

英文原词	中文译名	缩写／符号
Index	索引	
Model	模型	
Core	核心	
Hallucination	幻觉	
Embeddings	嵌入向量	
Inference	推理	
Benchmark	基准测试	
Embedding	嵌入	
Generator	生成器	
Reranker	重排序器	
Chunking	分块	
Accuracy	准确性	
Packing	打包	
Queries	查询	
Vector	向量	
Models	模型	
Token	词元	
Structure-aware Chunking	结构感知分块	
Fixed-size Chunking	固定大小分块	
Precision	精确率	
Baseline	基线	
Vectors	向量	
Recall	召回率	
Tokens	词元	
Value	值	V
Hypothetical Document Embeddings	假设文档嵌入	HyDE
Query Rewriting	查询改写	
Query Expansion	查询扩展	
Representation	表示	
Vocabulary	词表	
Key	键	K
Reciprocal Rank Fusion	倒数排名融合	RRF
Multi-hop Question	多跳问题	
Gating	门控	

英文原词	中文译名	缩写／符号
Poisoning	投毒	
Combine	合并	
Vector Database	向量数据库	
Training	训练	
Large Language Model	大语言模型	LLM
Prompt Injection	提示词注入	
Access Control	访问控制	
Routing	路由	

目录导航

点击章节标题跳转到中文译文，点击“原文”核对公式和图表。页码按本译本最终分页生成。

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

RAG & 上下文工程（Context Engineering） · 第 6 页 ［原文 1］

关于本手册 · 第 9 页 ［原文 2］

问题与解答（续，原文第 3 页） · 第 12 页 ［原文 3］

问题与解答（续，原文第 4 页） · 第 15 页 ［原文 4］

问题与解答（续，原文第 5 页） · 第 18 页 ［原文 5］

问题与解答（续，原文第 6 页） · 第 21 页 ［原文 6］

问题与解答（续，原文第 7 页） · 第 24 页 ［原文 7］

深度检查 · 第 27 页 ［原文 8］

RAG & 上下文工程（Context Engineering）

对应原文第 1 页；点击查看相邻原文对照页

RAG & 上下文工程（Context Engineering） 本手册中的问题

1. 什么是 检索增强生成（Retrieval-augmented Generation） (RAG)，它为语言 模型（Model） 添加了哪些 模型（Model） 参数单独无法提供的内容？

2. 上下文工程（Context Engineering） 比 提示词工程（Prompt Engineering） 范围更广，RAG 位于其中的什么位置？

3. 生产 RAG 流水线（Pipeline） 的主要阶段是什么？为什么要单独评估 解析（Parsing）、检索（Retrieval）、上下文组装（Context Assembly） 和 生成（Generation）？

4. 什么时候应该使用 RAG 而不是长上下文提示、微调（Fine-tuning） 或 继续预训练（Continued Pretraining）？

5. 为什么文档 解析（Parsing） 在 检索（Retrieval） 开始之前就会失败，特别是对于 PDF、表格和结构化文档？

6. 为什么 RAG 没有普遍正确的 块大小（Chunk Size）？

当块变小或变大时会发生什么变化？

7. 块重叠（Chunk Overlap） 实际上解决了什么问题，过多的重叠会如何伤害 检索（Retrieval） 和 生成（Generation）？

8. 在固定大小、结构感知和 语义分块（Semantic Chunking） 之间应该如何选择？

9. 上下文检索（Contextual Retrieval） 和 延迟分块（Late Chunking） 如何尝试保留文档级上下文，它们有何不同？

10. 什么是 层次检索（Hierarchical Retrieval），为什么类似 RAPTOR 的索引可以帮助处理长文档或需要多个抽象级别的问题？

11. 你应该如何选择top-k，为什么检索更多的段落会使最终的答案变得更糟？

12.为什么上下文窗口（Context Window）中检索到的证据（Evidence）的顺序和位置会改变模型（Model）性能（Performance）？

13. 重排序（Reranking）在检索（Retrieval）之后有什么贡献，为什么即使第一阶段检索器（Retriever）很强，它仍然有用？

14. 为什么要在 检索（Retrieval） 之前重写或扩展用户的 查询（Query），查询变换（Query Transformation） 如何会意外改变用户的意图？

15. 什么是 HyDE，为什么 模型（Model） 生成的假设文档决不能被视为 证据（Evidence）？

16. 多查询检索（Multi-query Retrieval） 和 排名融合（Rank Fusion） 何时可以提供帮助，为什么生成更多 查询（Query） 变体不会自动更好？

17. 为什么一次性 检索（Retrieval） 在多跳问题上会失败？将 检索（Retrieval） 与推理交错会发生什么变化？

18. 每个请求都应该检索固定数量的段落吗？

主动、适应性和纠正性 RAG 方法如何改变这一假设？

19.什么是上下文压缩（Context Compression）或选择性增强，压缩检索到的证据（Evidence）什么时候会降低系统的可靠性？

20. RAG 系统应如何处理检索到的冗余、不相关且相互冲突的 证据（Evidence）？

21. 当检索到的 证据（Evidence） 与 模型（Model） 的 参数化知识（Parametric Knowledge） 冲突时会发生什么？ 模型（Model） 将遵循哪个来源？

22. 为什么在答案中附加 引用（Citation） 并不能证明引用的来源支持该主张或 模型（Model） 实际上依赖于它？

23. 为什么即使 检索器（Retriever） 返回了正确的 证据（Evidence），RAG 系统也会产生幻觉？

24. 生产 RAG 系统应如何处理新鲜度、来源溯源（Provenance）、更新、删除和 重新建立索引（Re-indexing）？

25. 应如何在 RAG 中强制执行权限，以便未经授权的文档永远不会成为 模型（Model） 上下文？

26. 投毒或带有指令的文档如何攻击 RAG 系统？为什么检索到的文本应被视为不可信数据？

27. 您应该使用哪些指标来评估 检索（Retrieval）、上下文质量、生成（Generation）、归因和端到端 RAG 系统？

28. RAGAS、ARES 和 RAGChecker 实际上评估什么，为什么自动 RAG 判断不是 基准真值（Ground Truth）？

29. GraphRAG 旨在解决什么问题，以及什么时候基于图的 索引（Index） 是不必要的或者比普通 RAG 更糟糕？

30. 为不断变化的企业语料库设计一个生产 RAG 系统，包括权限、引用、多跳问题、长文档和严格的 延迟（Latency） SLO。你如何证明每一层都有效？

核心（Core） 概念、设计决策和故障模式

RAG & Context Engineering

Core Concepts, Design Decisions, and Failure Modes

@techNmak

Questions in this handbook

1. What is retrieval-augmented generation (RAG), and what does it add to a language model that the model's parameters alone do not provide?
2. How is context engineering broader than prompt engineering, and where does RAG fit inside it?
3. What are the major stages of a production RAG pipeline, and why should parsing, retrieval, context assembly, and generation be evaluated separately?
4. When should you use RAG instead of long-context prompting, fine-tuning, or continued pretraining?
5. Why can document parsing fail before retrieval even begins, especially for PDFs, tables, and structured documents?
6. Why is there no universally correct chunk size for RAG? What changes as chunks become smaller or larger?
7. What does chunk overlap actually solve, and how can too much overlap hurt retrieval and generation?
8. How should you choose between fixed-size, structure-aware, and semantic chunking?
9. How do Contextual Retrieval and Late Chunking try to preserve document-level context, and how are they different?
10. What is hierarchical retrieval, and why can RAPTOR-like indexes help with long documents or questions that require multiple levels of abstraction?
11. How should you choose top-k, and why can retrieving more passages make the final answer worse?
12. Why can the order and position of retrieved evidence inside the context window change model performance?
13. What does reranking contribute after retrieval, and why is it still useful even when the first-stage retriever is strong?
14. Why rewrite or expand a user's query before retrieval, and how can query transformation accidentally change the user's intent?
15. What is HyDE, and why must a hypothetical document generated by the model never be treated as evidence?
16. When can multi-query retrieval and rank fusion help, and why is generating more query variants not automatically better?
17. Why can one-shot retrieval fail on multi-hop questions, and what does interleaving retrieval with reasoning change?
18. Should every request retrieve a fixed number of passages? How do active, adaptive, and corrective RAG approaches change that assumption?
19. What is context compression or selective augmentation, and when can compressing retrieved evidence make the system less reliable?
20. How should a RAG system handle redundant, irrelevant, and mutually conflicting retrieved evidence?
21. What happens when retrieved evidence conflicts with the model's parametric knowledge? Which source will the model follow?
22. Why does attaching a citation to an answer not prove that the cited source supports the claim or that the model actually relied on it?
23. Why can a RAG system hallucinate even when the retriever returned the correct evidence?
24. How should a production RAG system handle freshness, provenance, updates, deletions, and re-indexing?
25. How should permissions be enforced in RAG so that unauthorized documents never become model context?
26. How can poisoned or instruction-bearing documents attack a RAG system, and why should retrieved text be treated as untrusted data?
27. Which metrics should you use to evaluate retrieval, context quality, generation, attribution, and the end-to-end RAG system?
28. What do RAGAS, ARES, and RAGChecker actually evaluate, and why are automated RAG judges not ground truth?
29. What problem is GraphRAG designed to solve, and when is a graph-based index unnecessary or worse than ordinary RAG?
30. Design a production RAG system for a changing enterprise corpus with permissions, citations, multi-hop questions, long documents, and a strict latency SLO. How would you prove each layer works?

关于本手册

对应原文第 2 页；点击查看相邻原文对照页

关于本手册 本手册重点介绍 检索增强生成（Retrieval-augmented Generation） 以及为语言 模型（Models） 提供的上下文的工程。它假设 嵌入向量（Embeddings）、词汇/密集 检索（Retrieval）、ANN 索引和 重排序（Reranking） 的机制分别涵盖。这里的目标是 检索（Retrieval） 之上的层：摄取、分块（Chunking）、查询变换（Query Transformation）、证据（Evidence） 选择、上下文 打包（Packing）、归因、安全性、评估和生产架构。个别论文的结果被表述为这些研究的结果，而不是普遍规律。

问题与解答

1. 什么是 检索增强生成（Retrieval-augmented Generation） (RAG)，它为 模型（Model） 语言添加了 模型（Model） 参数本身无法提供的什么内容？[初学者] RAG 使用在 推理（Inference） 时间从外部知识源检索到的信息来增强 生成（Generation）。

模型（Model） 仍然贡献其 参数化知识（Parametric Knowledge） 和语言 生成（Generation） 能力，但提示或 模型（Model） 计算以检索到的非参数 证据（Evidence） 为条件。在原始 RAG 公式中，检索器（Retriever） 为文档分配概率，并为输入和检索的文档分配 生成器（Generator） 条件；中心思想是有用的知识不必仅存储在 模型（Model） 权重中。这使得外部知识更容易更新，并为系统提供了通往 来源溯源（Provenance） 的路径。它不会自动使答案成为事实：检索器（Retriever） 可能会错过 证据（Evidence），语料库可能是错误的，而 生成器（Generator） 可能会忽略或误用正确的上下文。

因此，RAG 最好理解为 证据（Evidence） 访问架构，而不是 幻觉（Hallucination） 交换机。 [1]

2. 上下文工程（Context Engineering） 比 提示词工程（Prompt Engineering） 范围更广，RAG 位于其中的什么位置？

【中级】提示词工程（Prompt Engineering）主要是指令的编写和组织。 上下文工程（Context Engineering） 更广泛：它是在 推理（Inference） 时间呈现给 模型（Model） 的完整 词元（Token） 状态的设计 - 指令、检索到的 证据（Evidence）、工具结果、对话状态、元数据、示例和其他可能影响行为的信息。 RAG 是一种从外部语料库获取部分上下文的机制。如果检索到的段落顺序错误、冗余、过长、陈旧、未经授权或混有不明确的指令，那么一个好的 RAG 检索器（Retriever） 仍然会产生一个糟糕的系统。相反，上下文工程（Context Engineering） 可以在没有 向量（Vector） 检索（Retrieval） 的情况下存在；系统可以及时加载文件、使用结构化的 查询（Queries）、缓存的上下文或工具输出。实际目标不是最大化上下文长度。它提供一个足够小的高信号上下文，足以完成任务并且可供 模型（Model） 使用安全。 [2]

3. 生产 RAG 流水线（Pipeline） 的主要阶段是什么？为什么要单独评估 解析（Parsing）、检索（Retrieval）、上下文组装（Context Assembly） 和 生成（Generation）？ [中级 - 系统设计] 生产 流水线（Pipeline） 通常至少具有以下阶段：源摄取和解析（Parsing）；文档分割；

索引； 查询（Query）解释；候选项检索（Retrieval）；可选融合或重排序（Reranking）；授权过滤；上下文选择和排序； 生成（Generation）；归因;和记录/评估。每个阶段都会造成不同的失败。即使 检索（Retrieval） 是完美的，表也可能被错误地解析。候选集中可能不存在正确的段落。 重排序器（Reranker） 可能会丢弃它。该段落可能出现在提示中，但 生成器（Generator） 可能会忽略它。答案可能是正确的，但引用了错误的来源。端到端 准确性（Accuracy） 本身就隐藏了这些区别。例如，UDA 基准测试（

Benchmark） 明确研究 解析（Parsing）、检索（Retrieval）、生成（Generation） 和端到端文档分析作为单独的设计选择。工程启示是保留足够的中间工件 - 解析的单元、检索的 ID 和分数、重新排序的上下文、 最终提示、引用 - 来逐层测试 流水线（Pipeline）。 [3]

4. 什么时候应该使用 RAG 而不是长上下文提示、微调（Fine-tuning） 或 继续预训练（Continued Pretraining）？ [高级 - 判断] 当任务依赖的语料库太大、太动态、权限太敏感或 来源溯源（Provenance） 太关键而无法完全放入提示或 模型（Model） 权重时，请使用 RAG。当相关语料库足够小，可以经济地容纳并且 模型（Model） 可以可靠地使用它时，长上下文提示可以更简单。 微调（Fine-tuning） 更适合改变行为、风格、任务能力或学习模式；对于必须更新或引用的经常变化的事实记录来说，它是一个糟糕的替代品。 继续预训练（Continued Pretraining） 可以使 模型（Model） 适应域，但它也会更改权重并且不会给出文档级 来源溯源（Provenance）。不存在普遍的 RAG 与长上下文的胜利者。 2024 年的一项研究发现，在一些资源充足的环境下，长上下文平均更强，而 2025 年 LaRA 基准测试（Benchmark） 发现最佳选择取决于 模型（Model）、上下文长度、任务类型和 检索（Retrieval） 特征。基于测量的质量、成本、延迟（Latency）、新鲜度和治理要求的路线。 [26][27]

5. 为什么文档 解析（Parsing） 在 检索（Retrieval） 开始之前就会失败，特别是对于 PDF、表格和结构化文档？ [中级 - 生产] 检索（Retrieval） 只能搜索保留的摄取内容。真实的 PDF 和 Web 文档包含标题、栏目、脚注、表格、页面结构、阅读顺序歧义、扫描文本，有时还包含其结构在简单文本提取下消失的图形。因此，解析器可能会在 分块（Chunking） 或 嵌入（Embedding） 之前损坏 证据（Evidence）

About this handbook

This handbook focuses on retrieval-augmented generation and the engineering of the context supplied to language models. It assumes the mechanics of embeddings, lexical/dense retrieval, ANN indexes, and reranking are covered separately. The goal here is the layer above retrieval: ingestion, chunking, query transformation, evidence selection, context packing, attribution, security, evaluation, and production architecture. Results from individual papers are stated as results of those studies rather than universal laws.

Questions and answers

1. What is retrieval-augmented generation (RAG), and what does it add to a language model that the model's parameters alone do not provide? [Beginner]

RAG augments generation with information retrieved from an external knowledge source at inference time. The model still contributes its parametric knowledge and language-generation ability, but the prompt or model computation is conditioned on retrieved non-parametric evidence. In the original RAG formulation, the retriever assigns probabilities to documents and the generator conditions on both the input and retrieved document; the central idea is that useful knowledge need not be stored only in model weights. This makes external knowledge easier to update and gives the system a path to provenance. It does not make answers automatically factual: the retriever can miss the evidence, the corpus can be wrong, and the generator can ignore or misuse correct context. RAG is therefore best understood as an evidence-access architecture, not a hallucination switch. [1]

2. How is context engineering broader than prompt engineering, and where does RAG fit inside it? [Intermediate]

Prompt engineering is mainly about writing and organizing instructions. Context engineering is broader: it is the design of the complete token state presented to the model at inference time - instructions, retrieved evidence, tool results, conversation state, metadata, examples, and other information that may affect behavior. RAG is one mechanism for acquiring part of that context from an external corpus. A good RAG retriever can still produce a poor system if the retrieved passages are badly ordered, redundant, overlong, stale, unauthorized, or mixed with ambiguous instructions. Conversely, context engineering can exist without vector retrieval at all; a system may load files just in time, use structured queries, cached context, or tool outputs. The practical goal is not to maximize context length. It is to supply a small enough, high-signal context that is sufficient for the task and safe for the model to use. [2]

3. What are the major stages of a production RAG pipeline, and why should parsing, retrieval, context assembly, and generation be evaluated separately? [Intermediate - System Design]

A production pipeline commonly has at least these stages: source ingestion and parsing; document segmentation; indexing; query interpretation; candidate retrieval; optional fusion or reranking; authorization filtering; context selection and ordering; generation; attribution; and logging/evaluation. Each stage creates different failures. A table may be parsed incorrectly even if retrieval is perfect. The right passage may be absent from the candidate set. A reranker may discard it. The passage may be present in the prompt but the generator may ignore it. An answer may be correct yet cite the wrong source. End-to-end accuracy alone hides these distinctions. The UDA benchmark, for example, explicitly studies parsing, retrieval, generation, and end-to-end document analysis as separate design choices. The engineering lesson is to retain enough intermediate artifacts - parsed units, retrieved IDs and scores, reranked context, final prompt, citations - to test the pipeline layer by layer. [3]

4. When should you use RAG instead of long-context prompting, fine-tuning, or continued pretraining? [Advanced - Judgment]

Use RAG when the task depends on a corpus that is too large, too dynamic, too permission-sensitive, or too provenance-critical to place entirely in the prompt or model weights. Long-context prompting can be simpler when the relevant corpus is small enough to fit economically and the model can reliably use it. Fine-tuning is better suited to changing behavior, style, task competence, or learned patterns; it is a poor substitute for frequently changing factual records that must be updated or cited. Continued pretraining can adapt a model to a domain, but it also changes weights and does not give document-level provenance. There is no universal RAG-versus-long-context winner. A 2024 study found long context stronger on average under some sufficiently resourced settings, while the 2025 LaRA benchmark found the best choice depends on model, context length, task type, and retrieval characteristics. Route based on measured quality, cost, latency, freshness, and governance requirements. [26][27]

5. Why can document parsing fail before retrieval even begins, especially for PDFs, tables, and structured documents? [Intermediate - Production]

Retrieval can only search what ingestion preserved. Real PDFs and web documents contain headings, columns, footnotes, tables, page furniture, reading-order ambiguities, scanned text, and sometimes figures whose structure disappears under naive text extraction. A parser can therefore corrupt the evidence before chunking or embedding

问题与解答（续，原文第 3 页）

对应原文第 3 页；点击查看相邻原文对照页

发生。例如，转换为平面流的表可能会将 值（Value） 与其行和列标签分开；

检索（Retrieval） 稍后可能会返回看起来正确但含义错误的数字。 UDA 是围绕原始 PDF/HTML 设计的，并特别强调 解析（Parsing） 作为端到端 RAG 瓶颈。生产 系统应评估代表性文档类型的提取，保留有用的源坐标和结构元数据，并使用适合源的解析器或多模式方法处理表格或布局密集的材料。

更好的嵌入向量（Embeddings）无法恢复解析（Parsing）破坏的信息。 [3]

6. 为什么 RAG 没有普遍正确的 块大小（Chunk Size）？ 当块变小或变大时会发生什么变化？ [中级 - 权衡] 块大小（Chunk Size） 控制真正的权衡。非常小的块可以精确地定位事实并减少不相关的 词元（Tokens），但它们可以从它们解释的 证据（Evidence） 中切断定义、限定符、标题或多句推理。

大块保留更多的本地上下文，但它们的 嵌入向量（Embeddings） 可能会模糊多个主题，检索（Retrieval） 的选择性可能会降低，而 生成器（Generator） 会收到更多不相关的 词元（Tokens）。因此，最佳大小取决于文档结构、嵌入（Embedding） 模型（Model）、查询（Query） 粒度、检索（Retrieval） 策略以及预期答案的类型。最近的文档分割工作同样发现分割对 检索（Retrieval） 和 QA 产生重大影响，但没有建立一个通用的 词元（Token） 计数。可靠的工作流程是生成或标记有代表性的 查询（Queries）、扫描多个块大小/策略、测量 检索（Retrieval） 证据（Evidence） 覆盖范围和端到端答案质量，并检查边界故障。复制的默认值（例如“512 词元（Tokens）”）是起始配置，而不是工程结论。

7. 块重叠（Chunk Overlap） 实际上解决了什么问题，过多的重叠会如何伤害 检索（Retrieval） 和 生成（Generation）？ [中级] Overlap主要是一种边界对冲：如果一个重要的span跨越了一个chunk的边界，重复一些相邻的文本可以让至少一个chunk包含足够的证据（Evidence）。成本就是重复。更多的重叠会创建更多的 向量（Vectors） 或 索引（Index） 条目，增加摄取和存储，并且可以使几个几乎相同的块占据 top-k 结果。这会浪费上下文预算，并且可能会在 生成（Generation） 期间超重重复的 证据（Evidence）。它还可以使评估在块级别看起来更好，因为相同的事实出现多次，而无需增加独立的 证据（Evidence）。不存在始终最佳的通用百分比。当分割方法产生有害的边界切割时使用重叠，然后测量它是否真正改善了 证据（Evidence） 召回率（Recall）。结构感知分段、父子 检索（Retrieval）、上下文化块或在初始命中后检索相邻部分有时可以保持连续性并减少重复。目标是足够的 证据（Evidence），而不是最大的 词元（Token） 重用。

8. 在固定大小、结构感知和 语义分块（Semantic Chunking） 之间应该如何选择？ [高级 - 判断] 固定大小分块（Fixed-size Chunking） 便宜，确定性，并且是有用的 基线（Baseline）。 结构感知分块（Structure-aware Chunking） 遵循节、段落、代码块或表单元等边界，并且当源格式包含有意义的层次结构时通常更可取。 语义分块（Semantic Chunking） 尝试在主题发生变化的地方放置边界，但增加了 模型（Model） 或 嵌入（Embedding） 依赖性，并且不能保证在每个语料库上都优于更简单的方法。 ACL 2025 关于文档分段的研究结果表明，过大和过小的分段会产生不同的故障模式，并且分段可以改善 检索（Retrieval） 和端到端 QA，而 2026 年对学术文本的评估还发现，经过测试的基于集群的 语义分块（Semantic Chunking） 方法的性能并不优于更简单的固定大小或递归策略。按源结构和测量的 证据（Evidence） 检索（Retrieval） 进行选择。保留稳定的文档 ID 和父层次结构，以便系统可以检索 精确率（Precision） 的窄子级，同时在需要更广泛的上下文时仍扩展到其部分或文档。 [4][31]

9. 上下文检索（Contextual Retrieval） 和 延迟分块（Late Chunking） 如何尝试保留文档级上下文，它们有何不同？[高级] 两种技术都解决了相同的问题：对于看到整个文档的人来说，块可能在本地是可以理解的，但单独嵌入时却是模糊的。 Anthropic 的 上下文检索（Contextual Retrieval） 从完整文档中生成一个简短的特定于块的描述，并在构建语义和词汇索引之前添加该上下文。 延迟分块（Late Chunking） 采用不同的路线：长上下文 嵌入（Embedding） 模型（Model） 首先计算 词元（Token） 表示，而 词元（Tokens） 可以处理更大的文档，然后才将 词元（Token） 表示汇集到块 向量（Vectors） 中。前者改变了被索引的文本，可以与普通下游嵌入（Embedding）/BM25管道一起工作；后者改变了 分块（Chunking） 在 嵌入（Embedding） 计算中出现的位置，并且需要支持该方法的 模型（Model）。两者都不是普通 分块（Chunking） 的通用替代品。两者都增加了成本或 模型（Model） 假设，并且应该在实际语料库上进行评估。 Anthropic 报告的收益是其自己实验的结果，而不是一般保证。 [5][6]

10.什么是层次检索（Hierarchical Retrieval），为什么类RAPTOR索引可以帮助处理长文档或需要多个抽象级别的问题？[高级]扁平块检索（Retrieval）仅在一个粒度上表示长文档。 层次检索（Hierarchical Retrieval） 添加多个级别 -

抽象。 RAPTOR 递归地嵌入、聚类和总结文本以构建树，其较高节点

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

happens. For example, a table converted into a flat stream may separate a value from its row and column labels; retrieval may later return the correct-looking number with the wrong meaning. UDA was designed around raw PDFs/HTML and specifically highlights parsing as an end-to-end RAG bottleneck. Production systems should evaluate extraction on representative document types, preserve source coordinates and structural metadata where useful, and treat tables or layout-heavy material with parsers or multimodal methods appropriate to the source. Better embeddings cannot recover information that parsing destroyed. [3]

6. Why is there no universally correct chunk size for RAG? What changes as chunks become smaller or larger? [Intermediate - Trade-off]

Chunk size controls a real trade-off. Very small chunks can localize a fact precisely and reduce irrelevant tokens, but they can sever definitions, qualifiers, headers, or multi-sentence reasoning from the evidence they explain. Large chunks preserve more local context, but their embeddings can blur multiple topics, retrieval can become less selective, and the generator receives more irrelevant tokens. The best size therefore depends on document structure, embedding model, query granularity, retrieval strategy, and the kind of answer expected. Recent document-segmentation work likewise finds that segmentation materially affects retrieval and QA, but does not establish one universal token count. A defensible workflow is to generate or label representative queries, sweep several chunk sizes/strategies, measure retrieval evidence coverage and end-to-end answer quality, and inspect boundary failures. A copied default such as '512 tokens' is a starting configuration, not an engineering conclusion. [4]

7. What does chunk overlap actually solve, and how can too much overlap hurt retrieval and generation? [Intermediate]

Overlap is mainly a boundary hedge: if an important span crosses a chunk boundary, repeating some neighboring text can allow at least one chunk to contain enough of the evidence. The cost is duplication. More overlap creates more vectors or index entries, increases ingestion and storage, and can make several nearly identical chunks occupy the top-k results. That wastes context budget and can overweight repeated evidence during generation. It can also make evaluation look better at the chunk level because the same fact appears multiple times without increasing independent evidence. There is no universal percentage that is always optimal. Use overlap when your segmentation method creates harmful boundary cuts, then measure whether it actually improves evidence recall. Structure-aware segmentation, parent-child retrieval, contextualized chunks, or retrieving neighboring sections after the initial hit can sometimes preserve continuity with less duplication. The objective is sufficient evidence, not maximum token reuse.

8. How should you choose between fixed-size, structure-aware, and semantic chunking? [Advanced - Judgment]

Fixed-size chunking is cheap, deterministic, and a useful baseline. Structure-aware chunking follows boundaries such as sections, paragraphs, code blocks, or table units and is often preferable when the source format contains meaningful hierarchy. Semantic chunking tries to place boundaries where topics change, but adds model or embedding dependence and is not guaranteed to outperform simpler methods on every corpus. Findings ACL 2025 work on document segmentation shows that overly large and overly small segments create different failure modes and that segmentation can improve both retrieval and end-to-end QA, while a 2026 evaluation on academic texts also found a tested cluster-based semantic chunking method did not outperform simpler fixed-size or recursive strategies. Choose by source structure and measured evidence retrieval. Preserve stable document IDs and parent hierarchy so the system can retrieve a narrow child for precision while still expanding to its section or document when broader context is needed. [4][31]

9. How do Contextual Retrieval and Late Chunking try to preserve document-level context, and how are they different? [Advanced]

Both techniques address the same failure: a chunk may be locally understandable to a human who saw the whole document, yet ambiguous when embedded alone. Anthropic's Contextual Retrieval generates a short chunk-specific description from the full document and prepends that context before building semantic and lexical indexes. Late Chunking takes a different route: a long-context embedding model first computes token representations while the tokens can attend to the larger document, and only then are token representations pooled into chunk vectors. The former changes the text being indexed and can work with ordinary downstream embedding/BM25 pipelines; the latter changes where chunking occurs inside the embedding computation and requires a model that supports the method. Neither is a universal replacement for normal chunking. Both add cost or model assumptions and should be evaluated on the actual corpus. Anthropic's reported gains are results from its own experiments, not a general guarantee. [5][6]

10. What is hierarchical retrieval, and why can RAPTOR-like indexes help with long documents or questions that require multiple levels of abstraction? [Advanced]

Flat chunk retrieval represents a long document only at one granularity. Hierarchical retrieval adds multiple levels - for example, passages, sections, summaries, or clusters - so a query can retrieve either specific evidence or broader abstractions. RAPTOR recursively embeds, clusters, and summarizes text to build a tree whose higher nodes

问题与解答（续，原文第 4 页）

对应原文第 4 页；点击查看相邻原文对照页

代表越来越广泛的信息。在查询（Query）时间，它可以跨级别检索，旨在帮助需要整体或多步骤理解而不是局部跨度的问题。权衡是额外的索引成本和可能的摘要错误：抽象父节点本身是 模型（Model） 生成的，并且可以省略或扭曲细节。因此，常见的生产模式是使用更高级别的节点来导航或扩展 召回率（Recall），但保留指向 证据（Evidence） 和 引用（Citation） 原始源段落的链接。当文档结构或问题范围需要时，层次结构很有价值；对于以局部事实查找为主的小型语料库来说，这是不必要的复杂性。 [7]

11. 你应该如何选择top-k，为什么检索更多的段落会使最终的答案变得更糟？ [中级 - 上下文] Top-k 是上下文预算决策，而不是 检索（Retrieval） 常量。提高 k 可能会提高支持 证据（Evidence） 存在的机会，特别是当 检索器（Retriever） 的排名不完美时。但每一个额外的段落都可能添加不相关、冗余、冲突或质量较低的文本。这会增加投入成本，并可能使 证据（Evidence） 变得更加困难

固定数量的段落可能没有帮助。正确的 k 取决于候选 召回率（Recall） 曲线、重排序（Reranking） 质量、平均段落大小、模型（Model） 上下文行为和答案类型。在多个深度评估 检索（Retrieval） 召回率（Recall），然后使用实际的打包上下文评估 生成（Generation）。通常，正确的设计是廉价地检索更多候选者，对它们重新排序，删除重复项，并将较小的 证据（Evidence） 设置传递给 模型（Model），而不是简单地增加提示

12.为什么在上下文窗口（Context Window）中检索到的证据（Evidence）的顺序和位置会改变模型（Model） 性能（Performance）？ [中级 - 上下文] 长上下文 模型（Models） 不一定能同样好地使用所有位置。 Lost in the Middle 在多文档 QA 和 键（Key）-值（Value） 检索（Retrieval）: 模型（Models） 上显示出显着的位置敏感性，该研究中的相关信息在开始或结束附近出现时通常最强，而在中间出现时较弱。确切的模式是 模型（Model） 且与任务相关，因此不应将其变成通用规则，例如“始终将最好的块放在最后”。持久的教训是上下文 打包（Packing） 是 模型（Model） 界面的一部分。故意对最重要的 证据（Evidence） 进行排名，对相关段落进行分组，避免不必要的填充，并测试代表性任务的排列。重新排序还可能与引文、时间推理和矛盾来源相互作用。一个 模型（Model） 广告一个大的 上下文窗口（Context Window） 告诉您它可以接受多少个 词元（Tokens），而不是每个 词元（Token） 位置都会对答案质量做出同等的贡献。 [8]

13. 重排序（Reranking）在检索（Retrieval）之后有何贡献？为什么即使第一阶段检索器（Retriever）很强，它仍然有用？[中级-系统设计]第一阶段检索（Retrieval）经过优化，可以有效地搜索大型语料库，因此它经常使用更便宜的词汇或嵌入（Embedding）分数。 重排序（Reranking） 使用更强的相关性 模型（Model） 或更丰富的 查询（Query） 文档交互在更小的候选集上花费更多的计算。在 RAG 中，其 值（Value） 不仅仅是一个更高的搜索度量：它确定哪个 证据（Evidence） 消耗有限的 生成器（Generator） 上下文。更好的 重排序器（Reranker） 可以删除看似合理但错误的块，并将真正支持的段落移至提示中。权衡是额外的 延迟（Latency） 和成本，并且 重排序（Reranking） 仅帮助第一阶段检索的文档。因此，一个有用的设计是测量候选深度处的第一阶段 证据（Evidence） 召回率（Recall）、多个截止处的 重排序器（Reranker） 增益以及端到端答案质量。例如，Anthropic 的 上下文检索（Contextual Retrieval） 实验报告了添加 重排序（Reranking） 后 检索（Retrieval） 的额外改进，但这些数字特定于其评估的语料库和模型（Models）。 [5]

14. 为什么要在 检索（Retrieval） 之前重写或扩展用户的 查询（Query），查询变换（Query Transformation） 怎么会意外改变用户的意图？ [进阶] 用户 查询（Queries） 经常不明确、简短或用与语料库不同的 词表（Vocabulary） 表达。 查询扩展（Query Expansion） 或重写可以添加同义词、消除歧义的术语、可能的实体或更像文档的 表示（Representation）。例如，查询（Query）2doc 生成伪文档并使用它们来扩展 查询（Query），从而在其报告的实验中改进 检索（Retrieval）。风险在于语义漂移：语言 模型（Model） 可能会添加请求中没有的假设、错误地缩小不明确的 查询（Query） 范围，或者引入幻觉实体。一个强大的系统通常会保留原始的查询（Query），记录每个转换，并融合或比较结果，而不是默默地替换用户的意图。应根据重写改变含义的 召回率（Recall） 增益和失败情况来评估 查询改写（Query Rewriting）。它是一个 检索（Retrieval） 工具，没有重新解释用户任务的权限。 [9]

15.什么是 HyDE，为什么 模型（Model） 生成的假设文档决不能被视为 证据（Evidence）？ [高级] HyDE - 假设文档嵌入（Hypothetical Document Embeddings） - 要求语言 模型（Model） 生成一个假设文档，该文档将回答 查询（Query），嵌入生成的文本，并使用 嵌入（Embedding） 检索附近的真实文档。

生成的文档可能包含虚假详细信息； HyDE 论文明确承认它是“假的”。其作用是将 查询（Query） 移动到类似于相关文件的 表示（Representation），而不是提供事实的 证据（Evidence）。

答案 生成器（Generator） 仍应以检索到的语料库文档中的声明为依据。这种区别在操作上很重要：如果系统意外地将假设文本包含为可信上下文或引用它，则 检索（Retrieval） 辅助工具将成为幻觉来源。当标记的相关性数据不可用并且零样本密集 检索（Retrieval） 较弱时，HyDE 是最有趣的，但它的好处取决于工作负载，并且它在 检索（Retrieval） 之前添加了 LLM 调用。 [10]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

represent progressively broader information. At query time it can retrieve across levels, which was designed to help questions that require holistic or multi-step understanding rather than one local span. The trade-off is extra indexing cost and possible summary error: an abstractive parent node is itself model-generated and can omit or distort detail. A common production pattern is therefore to use higher-level nodes to navigate or broaden recall but retain links to original source passages for evidence and citation. Hierarchy is valuable when document structure or question scope demands it; it is unnecessary complexity for small corpora dominated by local fact lookup. [7]

11. How should you choose top-k, and why can retrieving more passages make the final answer worse? [Intermediate - Context]

Top-k is a context-budget decision, not a retrieval constant. Raising k may improve the chance that supporting evidence is present, especially when the retriever's ranking is imperfect. But every additional passage can add irrelevant, redundant, conflicting, or lower-quality text. That increases input cost and may make evidence harder for the generator to identify. Self-RAG specifically motivates adaptive retrieval partly because blindly retrieving a fixed number of passages can be unhelpful. The correct k depends on candidate recall curves, reranking quality, average passage size, model context behavior, and answer type. Evaluate retrieval recall at several depths and then evaluate generation with the actual packed context. Often the right design is to retrieve more candidates cheaply, rerank them, deduplicate them, and pass a smaller evidence set to the model rather than simply increasing prompt k. [14]

12. Why can the order and position of retrieved evidence inside the context window change model performance? [Intermediate - Context]

Long-context models do not necessarily use all positions equally well. Lost in the Middle showed substantial position sensitivity on multi-document QA and key-value retrieval: models in that study were often strongest when relevant information appeared near the beginning or end and weaker when it appeared in the middle. The exact pattern is model- and task-dependent, so it should not be turned into a universal rule such as 'always put the best chunk last.' The durable lesson is that context packing is part of the model interface. Rank the most important evidence deliberately, group related passages, avoid unnecessary filler, and test permutations on representative tasks. Reordering can also interact with citations, chronological reasoning, and contradictory sources. A model advertising a large context window tells you how many tokens it can accept, not that every token position will contribute equally to answer quality. [8]

13. What does reranking contribute after retrieval, and why is it still useful even when the first-stage retriever is strong? [Intermediate - System Design]

First-stage retrieval is optimized to search a large corpus efficiently, so it often uses cheaper lexical or embedding scores. Reranking spends more computation on a much smaller candidate set using a stronger relevance model or richer query-document interaction. In RAG, its value is not merely a higher search metric: it determines which evidence consumes the limited generator context. A better reranker can remove plausible-but-wrong chunks and move truly supporting passages into the prompt. The trade-off is extra latency and cost, and reranking only helps documents that the first stage retrieved. A useful design is therefore to measure first-stage evidence recall at candidate depth, reranker gains at several cutoffs, and end-to-end answer quality. Anthropic's Contextual Retrieval experiments, for example, reported additional retrieval improvements when reranking was added, but those numbers are specific to its evaluated corpora and models. [5]

14. Why rewrite or expand a user's query before retrieval, and how can query transformation accidentally change the user's intent? [Advanced]

User queries are often underspecified, short, or expressed with vocabulary that differs from the corpus. Query expansion or rewriting can add synonyms, disambiguating terms, likely entities, or a more document-like representation. Query2doc, for example, generates pseudo-documents and uses them to expand a query, improving retrieval in its reported experiments. The risk is semantic drift: the language model can add an assumption that was not in the request, narrow an ambiguous query incorrectly, or introduce a hallucinated entity. A robust system usually retains the original query, records every transformation, and fuses or compares results rather than silently replacing the user's intent. Query rewriting should be evaluated on both recall gains and failure cases where the rewrite changes meaning. It is a retrieval tool, not permission to reinterpret the user's task. [9]

15. What is HyDE, and why must a hypothetical document generated by the model never be treated as evidence? [Advanced]

HyDE - Hypothetical Document Embeddings - asks a language model to generate a hypothetical document that would answer the query, embeds that generated text, and uses the embedding to retrieve nearby real documents. The generated document can contain false details; the HyDE paper explicitly acknowledges that it is 'fake.' Its role is to move the query into a representation that resembles relevant documents, not to supply factual evidence. The answer generator should still ground claims in retrieved corpus documents. This distinction is operationally important: if a system accidentally includes the hypothetical text as trusted context or cites it, the retrieval aid becomes a hallucinated source. HyDE is most interesting when labeled relevance data are unavailable and zero-shot dense retrieval is weak, but its benefits are workload-dependent and it adds an LLM call before retrieval. [10]

问题与解答（续，原文第 5 页）

对应原文第 5 页；点击查看相邻原文对照页

16. 多查询检索（Multi-query Retrieval） 和 排名融合（Rank Fusion） 何时可以提供帮助，为什么生成更多的 查询（Query） 变体不会自动更好？[高级] 单个短语可能会错过相关材料，因此系统可能会生成多个 查询（Query） 变体，对每个变体进行检索，采用并集并融合排名。 倒数排名融合（Reciprocal Rank Fusion） 在这里很有用，因为它结合了排名位置，而不需要校准来自不同结果列表的原始分数。当变体捕获真正不同的 词表（Vocabulary） 或方面时，多查询检索（Multi-query Retrieval） 可以拓宽 召回率（Recall）。它还可以增加 延迟（Latency）、重复候选者，并在多个搜索中放大错误的重写。因此，更多的变体并不能单调地提高质量。

保留原始 查询（Query），删除重复结果，限制扩展预算，并测量每个添加的 查询（Query） 的边际 召回率（Recall）。如果语料库以精确标识符为主，或者原始 查询（Query） 已经是特定的，则额外的变体可能会增加大部分噪音。基本原理 - 查询扩展（Query Expansion） 加上 排名融合（Rank Fusion） - 比任何特定的“RAG-Fusion”配方更耐用。 [9][11]

17. 为什么一次性 检索（Retrieval） 在多跳问题上会失败，检索（Retrieval） 与推理交错会发生什么变化？ [高级] 对于 多跳问题（Multi-hop Question），第二次搜索所需的信息可能不会出现在原始问题中。

因此，即使语料库中存在所有必需的事实，检索（Retrieval） 调用也可能会失败。 IRCoT 演示了一种将检索（Retrieval） 与推理交错的方法：早期检索的 证据（Evidence） 和中间推理有助于制定下一步要检索的内容，而新的 证据（Evidence） 然后支持后面的推理步骤。架构教训是 查询（Query） 分解和有状态 检索（Retrieval），而不是思想链字符串本身必须公开或信任。生产系统可以将任务分解为子问题，检索每个子问题，维护显式的 证据（Evidence） 状态，并验证最终答案是否受到源联合的支持。迭代 检索（Retrieval） 需要更多调用，并且可能会级联早期错误的分解，因此仅当任务实际需要时才应调用多跳行为，并在跳级 证据（Evidence） 覆盖范围上进行评估。 [12]

18. 每个请求都应该检索固定数量的段落吗？主动、自适应和校正 RAG 方法如何改变这一假设？[高级] 修复了“为每个请求检索 top-k 一次”只是一个 RAG 策略。当即将出现的内容不确定时，FLARE 在长格式生成（Generation） 期间检索。 Self-RAG 训练 模型（Model） 发出反射信号，控制何时检索以及如何批评检索到的 证据（Evidence） 和生成。 CRAG 添加了一个 检索（Retrieval） 评估器，并在检索到的文档显得薄弱时选择纠正措施，包括知识精炼以及论文设置中的外部网络搜索。这些是不同的方法，但它们共享一个原则：检索（Retrieval） 可以是有条件的和反馈驱动的。权衡是复杂性。 门控（Gating） 模型（Model） 可能会犯自己的错误，迭代 检索（Retrieval） 会添加 延迟（Latency），而 Web 回退会改变 模型（Model） 的信任和新鲜度。当检索所有内容的成本很高或任务的知识需求差异很大时，请使用自适应 检索（Retrieval）；不要假设命名方法会原封不动地传输到每个系统。 [13][14][15]

19. 什么是上下文压缩（Context Compression）或选择性增强，什么时候压缩检索到的证据（Evidence）会使系统可靠性降低？[高级]检索到的段落可能比实际需要的证据（Evidence）长得多。 上下文压缩（Context Compression） 尝试保持高 值（Value） 含量，同时减少 词元（Tokens）。 RECOMP 训练提取或抽象压缩器，并在检索到的文档未添加有用信息时输出空字符串，将压缩与选择性增强相结合。 LongLLMLingua 压缩长提示以降低成本并解决信息密度/位置影响。失败模式很明显但很严重：压缩可以删除答案所依赖的限定符、数字、否定、异常或 来源溯源（Provenance） 提示。抽象压缩还可能在主 生

成器（Generator） 运行之前引入 生成（Generation） 错误。将压缩上下文视为派生的 表示（Representation）；保留原始段落以进行验证和引用，并在多个压缩比下评估答案质量。压缩只有好处

20. RAG 系统应如何处理检索到的冗余、不相关且相互冲突的 证据（Evidence）？ [高级-可靠性] 尽可能在最终提示之前处理冗余、无关和冲突。对几乎相同的块进行重复数据删除，这样重复的副本就不会消耗上下文或产生错误的证实感。重新排序或过滤不相关的候选者，而不是要求 生成器（Generator） 在嘈杂的提示中发现相关性。对于冲突的 证据（Evidence），保留源身份、时间戳和 来源溯源（Provenance）；不要默默地将矛盾合并到一个声明中。对冲突的 检索（Retrieval） 的研究表明，模型（Models） 对不一致的 证据（Evidence） 的反应很差，并且可能在特定的实验设置中表现出频率或确认效应。可靠的应用程序应该在重大冲突影响答案时将其暴露出来，根据明确的政策优先选择权威或当前的来源，并在冲突无法解决时允许弃权。“检索到的块的大多数”不是事实标准，因为重复或有毒的内容可以创建人为的多数。[18][19]

21. 当检索到的 证据（Evidence） 与 模型（Model） 的 参数化知识（Parametric Knowledge） 冲突时会发生什么？ 模型（Model） 将遵循哪个来源？[高级 - 可靠性] 没有通用的优先规则。根据 模型（Model）、提示、主题熟悉度、证据（Evidence） 质量和冲突设置，模型（Model） 可能会遵循检索到的上下文、坚持其参数先验、将两者混合或成为

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

16. When can multi-query retrieval and rank fusion help, and why is generating more query variants not automatically better? [Advanced]

A single phrasing can miss relevant material, so a system may generate several query variants, retrieve for each, take a union, and fuse the rankings. Reciprocal Rank Fusion is useful here because it combines rank positions without requiring raw scores from different result lists to be calibrated. Multi-query retrieval can broaden recall when variants capture genuinely different vocabulary or facets. It can also increase latency, duplicate candidates, and amplify a bad rewrite across several searches. More variants therefore do not monotonically improve quality. Preserve the original query, deduplicate results, cap the expansion budget, and measure marginal recall from each added query. If the corpus is dominated by exact identifiers or the original query is already specific, extra variants may add mostly noise. The underlying principles - query expansion plus rank fusion - are more durable than any particular 'RAG-Fusion' recipe. [9][11]

17. Why can one-shot retrieval fail on multi-hop questions, and what does interleaving retrieval with reasoning change? [Advanced]

For a multi-hop question, the information needed for the second search may not be present in the original question. One retrieval call can therefore fail even if every required fact exists in the corpus. IRCoT demonstrated a method that interleaves retrieval with reasoning: earlier retrieved evidence and intermediate reasoning help formulate what to retrieve next, and new evidence then supports later reasoning steps. The architectural lesson is query decomposition and stateful retrieval, not that a chain-of-thought string itself must be exposed or trusted. Production systems can decompose the task into subquestions, retrieve for each, maintain explicit evidence state, and verify that the final answer is supported by the union of sources. Iterative retrieval costs more calls and can cascade an early mistaken decomposition, so multi-hop behavior should be invoked only when the task actually requires it and evaluated on hop-level evidence coverage. [12]

18. Should every request retrieve a fixed number of passages? How do active, adaptive, and corrective RAG approaches change that assumption? [Advanced]

Fixed 'retrieve top-k once for every request' is only one RAG policy. FLARE retrieves during long-form generation when upcoming content appears uncertain. Self-RAG trains a model to emit reflection signals that control when to retrieve and how to critique retrieved evidence and generations. CRAG adds a retrieval evaluator and chooses corrective actions when retrieved documents appear weak, including knowledge refinement and, in the paper's setup, external web search. These are different methods, but they share a principle: retrieval can be conditional and feedback-driven. The trade-off is complexity. A gating model can make its own mistakes, iterative retrieval adds latency, and web fallback changes the trust and freshness model. Use adaptive retrieval when the cost of retrieving everything is high or when tasks vary substantially in knowledge needs; do not assume a named method transfers unchanged to every system. [13][14][15]

19. What is context compression or selective augmentation, and when can compressing retrieved evidence make the system less reliable? [Advanced]

Retrieved passages can be much longer than the evidence actually needed. Context compression tries to keep high-value content while reducing tokens. RECOMP trains extractive or abstractive compressors and can output an empty string when retrieved documents add no useful information, combining compression with selective augmentation. LongLLMLingua compresses long prompts to reduce cost and address information density/position effects. The failure mode is obvious but serious: compression can delete a qualifier, number, negation, exception, or provenance cue that the answer depends on. Abstractive compression can also introduce generation error before the main generator runs. Treat compressed context as a derived representation; retain the original passages for verification and citations, and evaluate answer quality at several compression ratios. Compression is beneficial only if the saved tokens and latency outweigh preprocessing cost and lost evidence. [16][17]

20. How should a RAG system handle redundant, irrelevant, and mutually conflicting retrieved evidence? [Advanced - Reliability]

Redundancy, irrelevance, and conflict should be handled before the final prompt where possible. Deduplicate near-identical chunks so repeated copies do not consume context or create a false sense of corroboration. Rerank or filter irrelevant candidates rather than asking the generator to discover relevance inside a noisy prompt. For conflicting evidence, preserve source identity, timestamp, and provenance; do not silently merge contradictions into one statement. Research on conflicting retrieval shows that models can respond poorly to inconsistent evidence and may exhibit frequency or confirmation effects in particular experimental settings. A reliable application should surface material conflicts when they affect the answer, prefer authoritative or current sources according to explicit policy, and allow abstention when conflict cannot be resolved. 'Majority of retrieved chunks' is not a truth criterion because duplicate or poisoned content can create an artificial majority. [18][19]

21. What happens when retrieved evidence conflicts with the model's parametric knowledge? Which source will the model follow? [Advanced - Reliability]

There is no universal precedence rule. Depending on the model, prompt, topic familiarity, evidence quality, and conflict setup, a model may follow retrieved context, stick to its parametric prior, blend the two, or become

问题与解答（续，原文第 6 页）

对应原文第 6 页；点击查看相邻原文对照页

不一致。知识冲突的研究观察到在不同条件下对外部 证据（Evidence） 的过度依赖和不正确的内部知识的持续存在。工程含义是显式编码源策略并对其进行测试。如果应用程序要求提供的语料库具有权威性，请明确说明，包括 来源溯源（Provenance），并评估 模型（Model） 记忆的答案不同的反事实情况。如果语料库本身可能是错误的，那么盲目的上下文服从也是不安全的。对于高风险冲突，返回分歧或寻求其他来源，而不是强迫 模型（Model） 选择一个。 RAG 将可用的 证据（Evidence） 更改为 模型（Model）；它不会自动校准 模型（Model） 如何仲裁 证据（Evidence）。 [18][19]

22. 为什么在答案中附加 引用（Citation） 并不能证明引用的来源支持该主张或 模型（Model） 实际上依赖于它？ [高级 - 归因] 引用（Citation） 可能会以多种不同的方式失败。所引用的段落可能不包含主张；索赔可能根本没有 引用（Citation）； 引用（Citation） 可能仅支持复合句的一部分；或者 模型（Model） 在从参数存储器生成答案后可能会生成看起来正确的 引用（Citation）。 ALCE 评估引用的 生成（Generation） 并将答案质量与 引用（Citation） 质量分开。后来的工作区分了 引用（Citation） 的正确性 - 来源是否支持该声明 -

在生产中，保留从声明到源跨度的映射，检查 引用（Citation） 蕴含或支持，测量外部可验证声明的覆盖范围，并使用户可以轻松开放源。句子旁边的 URL 是元数据，而不是证据。归因是其自身的评价目标。 [20][21]

23. 为什么即使 检索器（Retriever） 返回了正确的 证据（Evidence），RAG 系统也会产生幻觉？

[中级 - 可靠性] 正确的 检索（Retrieval） 仅解决了 证据（Evidence） 访问问题。 生成器（Generator） 仍然可能忽略该段落，误读表格，犯算术或推理错误，合并（Combine） 证据（Evidence） 不正确，遵循不相关的相邻段落，或者更喜欢其参数先验。它还可以围绕其他受支持的 核心（Core） 答案生成不受支持的详细信息。

这就是为什么 RAG 评估框架将上下文相关性与答案忠实性/相关性/正确性分开，而不是将 检索（Retrieval） 召回率（Recall） 视为最终指标。一个有用的调试测试是 Oracle 上下文实验：给出仅已知支持 证据（Evidence） 的 生成器（Generator）。如果答案仍然失败，检索（Retrieval） 不再是主要瓶颈。

相反，如果 oracle 上下文工作但正常 RAG 失败，请检查 解析（Parsing）、检索（Retrieval）、重排序（Reranking） 或 打包（Packing）。

RAG 可以减少一些事实错误，但它并不能消除将 生成器（Generator） 作为推理和 证据（Evidence） 使用组件进行评估的需要。 [22][23][24]

24. 生产 RAG 系统应如何处理新鲜度、来源溯源（Provenance）、更新、删除和 重新建立索引（Re-indexing）？ [高级 - 生产] RAG 仅与 索引（Index） 流水线（Pipeline） 的源一样新鲜。存储稳定的源 ID、版本或修订标识符、时间戳、来源溯源（Provenance） 和摄取状态。更新应该传播到所有派生的工件：解析的文本、块、嵌入向量（Embeddings）、词汇索引、图形结构、缓存和 引用（Citation） 映射。删除尤其重要，因为删除源记录但保留旧的 嵌入（Embedding） 或缓存的提示可能会继续暴露过时或禁止的文本。根据新鲜度 SLO 确定更新是事件驱动的、批量的还是定期的，并明确测量摄取延迟。在迁移期间，支持双索引或版本化部署，以便 查询（Queries） 不会混合不兼容的表示形式。非参数记忆最初的吸

引力在于无需重新训练 生成器（Generator） 即可更新知识；仅当工程系统真正保持 索引（Index） 的新鲜度和血统时，这种优势才存在。 [1]

25. 应如何在 RAG 中强制执行权限，以便未经授权的文档永远不会成为 模型（Model） 上下文？ [高级 - 安全性] 授权必须是数据访问边界，而不是 模型（Model） 的自然语言指令。在将未经授权的内容组装到 模型（Model） 上下文中之前，系统应确定用户的有效权限并限制候选文档。根据架构的不同，这可能意味着在搜索层中进行权限感知过滤，以调用者的身份查询源系统，或者在提示构建之前进行 检索（Retrieval） 后授权 - 但绝不依赖 LLM 来“忽略”不允许其查看的文档。权限还必须传播到缓存的结果、摘要、图形节点和派生索引。最近的企业访问控制研究主张在 检索（Retrieval） 和 生成（Generation） 边界上执行确定性执行，因为即时清理和输出过滤是概率性保障措施，不能替代授权。记录哪些受保护对象影响每个答案，以便访问决策可审核。 [30]

26. 投毒或带有指令的文档如何攻击 RAG 系统？为什么检索到的文本应被视为不可信数据？ [高级 - 安全] RAG 语料库扩大了攻击面，因为检索到的文本是外部输入。 PoisonedRAG 证明，将恶意文档注入知识库可以引导针对目标 查询（Queries） 的答案，并且后来的安全基准测试显示了具有误导性或对抗性上下文的更广泛的稳健性问题。即使没有优化的 投毒（Poisoning） 攻击，普通文档也可能包含与系统策略冲突的类似指令的文本。因此，检索到的内容应被视为不受信任的数据：保留 来源溯源（Provenance） 和信任级别、单独的指令

在适当的情况下，强制执行 模型（Model） 之外的工具权限，并监视可疑的 检索（Retrieval） 模式。不

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

inconsistent. Studies of knowledge conflict have observed both over-reliance on external evidence and persistence of incorrect internal knowledge under different conditions. The engineering implication is to encode source policy explicitly and test it. If the application requires the provided corpus to be authoritative, say so, include provenance, and evaluate counterfactual cases where the model's memorized answer differs. If the corpus itself can be wrong, blind context obedience is also unsafe. For high-stakes conflicts, return the disagreement or seek another source rather than forcing the model to pick one. RAG changes the evidence available to the model; it does not automatically calibrate how the model arbitrates evidence. [18][19]

22. Why does attaching a citation to an answer not prove that the cited source supports the claim or that the model actually relied on it? [Advanced - Attribution]

A citation can fail in several different ways. The cited passage may not entail the claim; a claim may have no citation at all; the citation may support only part of a compound sentence; or the model may generate a correct-looking citation after producing an answer from parametric memory. ALCE evaluates cited generation and separates answer quality from citation quality. Later work distinguishes citation correctness - whether the source supports the claim - from citation faithfulness - whether the model genuinely relied on the cited evidence rather than post-rationalizing. In production, preserve a mapping from claims to source spans, check citation entailment or support, measure coverage of externally verifiable claims, and make source opening easy for users. A URL next to a sentence is metadata, not proof. Attribution is its own evaluation target. [20][21]

23. Why can a RAG system hallucinate even when the retriever returned the correct evidence? [Intermediate - Reliability]

Correct retrieval solves only the evidence-access problem. The generator can still overlook the passage, misread a table, make an arithmetic or reasoning error, combine evidence incorrectly, follow an irrelevant neighboring passage, or prefer its parametric prior. It can also generate unsupported details around an otherwise supported core answer. This is why RAG evaluation frameworks separate context relevance from answer faithfulness/relevance/correctness rather than treating retrieval recall as the final metric. A useful debugging test is an oracle-context experiment: give the generator only known supporting evidence. If the answer still fails, retrieval is no longer the main bottleneck. Conversely, if the oracle context works but normal RAG fails, inspect parsing, retrieval, reranking, or packing. RAG can reduce some factual errors, but it does not remove the need to evaluate the generator as a reasoning and evidence-use component. [22][23][24]

24. How should a production RAG system handle freshness, provenance, updates, deletions, and re-indexing? [Advanced - Production]

RAG is only as fresh as the source-to-index pipeline. Store stable source IDs, version or revision identifiers, timestamps, provenance, and ingestion status. Updates should propagate to all derived artifacts: parsed text, chunks, embeddings, lexical indexes, graph structures, caches, and citation mappings. Deletion is especially important because removing a source record but leaving an old embedding or cached prompt can continue to expose stale or prohibited text. Decide whether updates are event-driven, batched, or periodic based on the freshness SLO, and measure ingestion lag explicitly. During migrations, support dual indexes or versioned rollouts so queries do not mix incompatible representations. The original attraction of non-parametric memory was that knowledge can be updated without retraining the generator; that advantage only exists if the engineering system actually maintains index freshness and lineage. [1]

25. How should permissions be enforced in RAG so that unauthorized documents never become model context? [Advanced - Security]

Authorization must be a data-access boundary, not a natural-language instruction to the model. The system should determine the user's effective permissions and restrict candidate documents before unauthorized content is assembled into model context. Depending on the architecture, that can mean permission-aware filtering in the search layer, querying source systems under the caller's identity, or post-retrieval authorization before prompt construction - but never relying on the LLM to 'ignore' a document it was not allowed to see. Permissions must also propagate to cached results, summaries, graph nodes, and derived indexes. Recent enterprise access-control research argues for deterministic enforcement at both retrieval and generation boundaries because prompt sanitization and output filtering are probabilistic safeguards, not substitutes for authorization. Log which protected objects influenced each answer so access decisions are auditable. [30]

26. How can poisoned or instruction-bearing documents attack a RAG system, and why should retrieved text be treated as untrusted data? [Advanced - Security]

A RAG corpus expands the attack surface because retrieved text is external input. PoisonedRAG demonstrated that injecting malicious documents into a knowledge base can steer answers for targeted queries, and later security benchmarks show broader robustness problems with misleading or adversarial context. Even without an optimized poisoning attack, ordinary documents can contain instruction-like text that conflicts with system policy. Retrieved content should therefore be treated as untrusted data: preserve provenance and trust level, separate instructions from evidence in prompt design, restrict which sources can enter sensitive workflows, scan or validate ingestion where appropriate, enforce tool permissions outside the model, and monitor suspicious retrieval patterns. No

问题与解答（续，原文第 7 页）

对应原文第 7 页；点击查看相邻原文对照页

内容过滤器是安全性的证明，因此高影响力的操作应该具有确定性的授权和验证门。

模型（Model） 不应仅仅因为检索到的文档告诉它而获得新的权限。间接提示注入研究专门研究通过外部内容而不是直接来自用户的恶意指令。 [28][29][32]

27. 您应该使用哪些指标来评估 检索（Retrieval）、上下文质量、生成（Generation）、归因和端到端 RAG 系统？ [高级 - 评估] 多层评估。对于 检索（Retrieval），在合适的情况下使用面向 证据（Evidence） 的测量，例如 召回率（Recall）@k、精确率（Precision）@k、MRR/nDCG，加上 延迟（Latency） 和真实 查询（Query） 类型的覆盖范围。对于组装的上下文，测量是否存在支持 证据（Evidence）、剩余多少不相关或重复的材料以及所需的权限/来源溯源（Provenance） 是否正确。对于 生成（Generation），区分答案的正确性、对提供的 证据（Evidence） 的忠实度以及答案的相关性。对于归因答案，请测量 引用（Citation） 支持度和 引用（Citation） 完整性。端到端评估还应包括弃权行为、新鲜度、安全案例、延迟（Latency）、成本和特定于任务的业务成果。单个总分无法告诉您要修复哪个模块。为困难类别（例如多跳、精确标识符、冲突源、陈旧文档、表格和权限过滤的 查询（Queries））构建切片级结果。 RAGAS、ARES 和 RAGChecker 都反映了这种多维视图。 [22][23][24]

28. RAGAS、ARES 和 RAGChecker 实际上评估什么，为什么自动 RAG 判断不是 基准真值（Ground Truth）？ [高级 - 评估] RAGAS 使用自动评估提出了上下文质量、答案忠实度和答案相关性等维度的无参考指标。 ARES 生成合成的 训练（Training） 数据，训练轻量级法官的上下文相关性、答案忠实度和答案相关性，并使用小型人工注释验证集和预测驱动的 推理（Inference） 来改进总体估计和置信区间。 RAGChecker 使用声明级操作以及更大的 检索器（Retriever） 和 生成器（Generator） 诊断指标套件，然后元评估与人类判断的相关性。它们是有用的工具，而不是可互换的真理机器。基于 LLM 的法官继承了 模型（Model） 偏见、提示敏感性、领域不匹配和错误。综合问题可能会歪曲生产流量。使用自动评估迭代速度，但根据真实领域中人工标记的示例对其进行校准，并定期检查分歧。仅当其测量行为已针对您的用例进行验证时，指标才值得信赖。 [22][23][24]

29. GraphRAG 旨在解决什么问题，基于图的 索引（Index） 何时是不必要的或比普通 RAG 更糟糕？ [高级] GraphRAG 的目标问题的答案取决于语料库中的关系或全局模式，而不是一两个局部相似的段落。 Microsoft 的 GraphRAG 构建实体/关系图，将其聚类为社区，并创建社区摘要；然后，全局搜索可以对这些摘要进行推理，而本地模式则使用以实体为中心的上下文。已发表的作品特别关注语料库级别的意义建构问题，而传统的本地 RAG 并不适合这些问题。 GraphRAG 不是通用升级。索引需要实体/关系提取和 LLM 生成的摘要，这会增加成本、延迟（Latency）、存储和新的错误模式。对于直接事实查找、精确标识符或小型文档集合，普通词汇/密集 检索（Retrieval） 可能更简单、更好。当 查询（Query） 真正需要关系、主题或多文档聚合时，请使用图形结构，并保留返回原始 证据（Evidence） 的链接，因为图形节点和摘要是派生工件。 [25]

30. 为不断变化的企业语料库设计一个生产 RAG 系统，包括权限、引用、多跳问题、长文档和严格的 延迟（Latency） SLO。你如何证明每一层都有效？ [高级 - 系统设计] 从治理和工作负载开始，而不是 向量数据库（Vector Database）。摄取应保留源 ID、版本、结构和 ACL； 解析（Parsing） 必须在代表性 PDF/表格上进行测试。构建一个简单的词汇+密集 基线（Baseline），测量 证据（Evidence） 召回率（

Recall），然后仅在改进候选到上下文步骤时添加 重排序（Reranking）。从文档结构和评估中选择 分块（Chunking），而不是固定的 词元（Token） 配方；仅使用分层或上下文化的 检索（Retrieval）

检索（Retrieval）；对于语料库级别的主题问题，考虑图表/分层方法；对于小型有界语料库，直接测试长上下文。上下文 打包（Packing） 应删除重复项、保留 来源溯源（Provenance）、故意订购高 值（Value） 证据（Evidence），并适合明确的 词元（Token） 预算。 生成器（Generator） 应引用来源跨度，并在 证据（Evidence） 不充分或冲突时弃权。安全测试必须包括投毒/带有指令的文档和权限边界。通过特定于层的评估以及端到端切片、延迟（Latency）/成本 SLO、新鲜度滞后和事件记录来证明系统。每个添加的组件都需要一个可测量的故障模式来修复；否则就是建筑装饰。 [3][25][26][27][30]

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

content filter is a proof of safety, so high-impact actions should have deterministic authorization and validation gates. The model should not gain new privileges merely because a retrieved document told it to. Indirect prompt-injection research specifically studies malicious instructions arriving through external content rather than directly from the user. [28][29][32]

27. Which metrics should you use to evaluate retrieval, context quality, generation, attribution, and the end-to-end RAG system? [Advanced - Evaluation]

Evaluate at multiple layers. For retrieval, use evidence-oriented measures such as Recall@k, Precision@k, MRR/nDCG where suitable, plus latency and coverage on real query types. For the assembled context, measure whether supporting evidence is present, how much irrelevant or duplicate material remains, and whether required permissions/provenance are correct. For generation, distinguish answer correctness from faithfulness to supplied evidence and from answer relevance. For attributed answers, measure citation support and citation completeness. End-to-end evaluation should also include abstention behavior, freshness, security cases, latency, cost, and task-specific business outcomes. A single aggregate score cannot tell you which module to fix. Build slice-level results for difficult categories such as multi-hop, exact identifiers, conflicting sources, stale documents, tables, and permission-filtered queries. RAGAS, ARES, and RAGChecker all reflect this multi-dimensional view. [22][23][24]

28. What do RAGAS, ARES, and RAGChecker actually evaluate, and why are automated RAG judges not ground truth? [Advanced - Evaluation]

RAGAS proposes reference-free metrics for dimensions such as context quality, answer faithfulness, and answer relevance using automated evaluation. ARES generates synthetic training data, trains lightweight judges for context relevance, answer faithfulness, and answer relevance, and uses a small human-annotated validation set with prediction-powered inference to improve aggregate estimates and confidence intervals. RAGChecker uses claim-level operations and a larger suite of retriever and generator diagnostic metrics, then meta-evaluates correlation with human judgments. They are useful tools, not interchangeable truth machines. LLM-based judges inherit model bias, prompt sensitivity, domain mismatch, and error. Synthetic questions can misrepresent production traffic. Use automated evaluation for iteration speed, but calibrate it against human-labeled examples from the real domain and periodically inspect disagreements. A metric is trustworthy only to the extent that its measurement behavior has been validated for your use case. [22][23][24]

29. What problem is GraphRAG designed to solve, and when is a graph-based index unnecessary or worse than ordinary RAG? [Advanced]

GraphRAG targets questions whose answer depends on relationships or global patterns across a corpus rather than one or two locally similar passages. Microsoft's GraphRAG builds an entity/relationship graph, clusters it into communities, and creates community summaries; global search can then reason over those summaries, while local modes use entity-centered context. The published work focuses particularly on corpus-level sensemaking questions where conventional local RAG is a poor fit. GraphRAG is not a universal upgrade. Indexing requires entity/relationship extraction and LLM-generated summaries, which adds cost, latency, storage, and new error modes. For direct fact lookup, exact identifiers, or small document collections, normal lexical/dense retrieval may be simpler and better. Use graph structure when the query genuinely needs relations, themes, or multi-document aggregation, and keep links back to original evidence because graph nodes and summaries are derived artifacts. [25]

30. Design a production RAG system for a changing enterprise corpus with permissions, citations, multi-hop questions, long documents, and a strict latency SLO. How would you prove each layer works? [Advanced - System Design]

Start with governance and workload, not a vector database. Ingestion should preserve source IDs, versions, structure, and ACLs; parsing must be tested on representative PDFs/tables. Build a simple lexical+dense baseline, measure evidence recall, then add reranking only if it improves the candidate-to-context step. Choose chunking from document structure and evaluation rather than a fixed token recipe; use hierarchical or contextualized retrieval only for failure cases they solve. Apply authorization before prompt assembly. For multi-hop questions, route to iterative retrieval; for corpus-level thematic questions, consider graph/hierarchical methods; for small bounded corpora, test long-context directly. Context packing should deduplicate, preserve provenance, order high-value evidence deliberately, and fit an explicit token budget. The generator should cite source spans and abstain when evidence is insufficient or conflicting. Security tests must include poisoned/instruction-bearing documents and permission boundaries. Prove the system with layer-specific evals plus end-to-end slices, latency/cost SLOs, freshness lag, and incident logging. Every added component needs a measured failure mode it fixes; otherwise it is architectural decoration. [3][25][26][27][30]

深度检查

对应原文第 8 页；点击查看相邻原文对照页

深度检查区域什么是强理解 分块（Chunking） 解释了 证据（Evidence） 粒度权衡，并提出了特定于语料库的评估，而不是命名神奇的 词元（Token） 大小。

上下文 打包（Packing） 将 top-k、重复数据删除、排序和压缩视为 词元（Token） 预算优化问题，而不是独立切换。

可靠性 将 检索（Retrieval） 故障与 生成（Generation） 故障区分开来，并了解正确的上下文仍然可以产生不受支持的答案。

归因区分 证据（Evidence） 的答案正确性、支持性、引用覆盖率和真实性。

安全性 在 上下文组装（Context Assembly） 之前强制执行授权，并将检索到的文档视为不受信任的输入，而不是指令。

评估使用特定于层的指标和代表性切片，然后根据人工标记的生产示例验证自动判断。

值得了解的其他概念提示缓存和上下文重用。有时可以缓存重复的大型上下文以降低成本或 延迟（Latency），但缓存身份和授权成为正确性的一部分。

即时代理搜索。代理可以保留轻量级引用并仅在需要时加载信息，而不是预加载所有潜在相关的上下文；这属于更广泛的上下文工程问题。

结构化和多模式 RAG。 SQL、知识图、表格、图像和代码通常需要 检索（Retrieval） 或工具接口来保留结构，而不是将所有内容展平为文本块。

弃权和问责。生产系统应区分“未找到相关的 证据（Evidence）”和“找到 证据（Evidence） 但不足/冲突”，并测试 模型（Model） 是否适当拒绝。

参考文献

[1] 帕特里克·刘易斯等人。 检索增强生成（Retrieval-augmented Generation） 适用于知识密集型 NLP 任务。 NeurIPS 2020。

[2] 人择。对 AI 代理有效的 上下文工程（Context Engineering）。工程博客，2025 年。

[3] 辉玉龙，路姚，张焕辰。 UDA：基准测试（Benchmark） 套件

NeurIPS 2024 数据集和基准跟踪。

[4] 王志同等.文档分段对于 检索（Retrieval）-增强型 生成（Generation） 很重要。 ACL 2025 的调查结果。

[5] 人择。介绍 上下文检索（Contextual Retrieval）。工程博客，

[6] 迈克尔·冈瑟、伊莎贝尔·莫尔、丹尼尔·詹姆斯·威廉姆斯、王博、韩晓。 延迟分块（Late Chunking）：上下文块 嵌入向量（Embeddings） 使用长上下文 嵌入（Embedding） 模型（Models）。 arXiv：2409.04701，2024。

[7] 帕斯·萨蒂等人。 RAPTOR：树形组织 检索（Retrieval） 的递归抽象处理。 ICLR 2024。

[8] 纳尔逊·刘等人。迷失在中间：语言 模型（Models） 如何使用长上下文。 TACL 2023。

[9] 王亮，杨南，魏福如。 查询（Query）2doc：查询扩展（Query Expansion） 与大语言 模型（Models）。 EMNLP 2023。

[10] 高鲁宇，马雪光，林志颖，杰米·卡兰。无相关标签 (HyDE) 的精确零射击密集 检索（Retrieval）。前交叉韧带

[11] 戈登·V·科马克、查尔斯·L·A·克拉克、斯特凡·布彻。 倒数排名融合（Reciprocal Rank Fusion） 优于孔多塞和个人排名学习方法。西吉尔 2009。

[12] 哈什·特里维迪等人。将 检索（Retrieval） 与知识密集型多步骤问题的思想链推理交织在一起。访问控制2023。

[13]蒋正宝等.活动 检索（Retrieval） 增强型 生成（Generation） (FLARE)。 EMNLP 2023。

[14] 浅井明里等人。 Self-RAG：通过自我反思学习检索、生成和批评。 ICLR 2024。

[15] 严世奇，顾嘉晨，朱云，凌振华。修正 检索（Retrieval） 增强 生成（Generation）。 arXiv：2401.15884，2024。

[16] 徐方圆，施维佳，崔恩索尔. RECOMP：使用 上下文压缩（Context Compression） 和选择性增强改进 检索（Retrieval） 增强型 LM。 ICLR 2024。

[17]蒋惠强等. LongLLMLingua：通过即时压缩在长上下文场景中加速和增强法学硕士。前交叉韧带

[18] 陈鸿廷，张迈克尔，崔恩索尔。丰富的知识来源带来复杂的知识冲突：重新校准模型（Models）以反映冲突的证据（Evidence）。 EMNLP 2022。

[19] 金卓然等.知识之间的拉锯战：检索（Retrieval）增强语言模型（Models）中知识冲突的探索和解决。 LREC-冷却 2024。

[20] 高天宇，甄子丹，于佳桐，陈丹琪。启用大语言 模型（Models） 生成带引文的文本 (ALCE)。

EMNLP 2023。

[21] 乔纳斯·瓦拉特、玛丽亚·赫斯、马丁·德·里克、阿维舍克·阿南德。

正确性并不是 RAG 归因中的忠实性。 ICTIR 2025。

[22] Shahul Es、Jithin James、Luis Espinosa-Anke、Steven Schockaert。

RAGAS：检索（Retrieval） 增强型 生成（Generation） 的自动评估。

EACL 2024 系统演示。

[23] 乔恩·萨德-法尔肯、奥马尔·哈塔布、克里斯托弗·波茨、马泰·扎哈里亚。

ARES：检索增强生成（Retrieval-augmented Generation） 系统的自动评估框架。 NAACL 2024。

[24] 茹冬雨等. RAGChecker：用于诊断 检索增强生成（Retrieval-augmented Generation） 的细粒度框架。 NeurIPS 2024 数据集和基准跟踪。

[25]达伦·埃奇等人。从局部到全局：查询（Query） 聚焦摘要的图 RAG 方法。 arXiv：2404.16130，2025 年修订。

[26] 李卓万，李成，张明阳，梅巧竹，Michael Bendersky。 检索（Retrieval） 增强型 生成（Generation） 还是长上下文法学硕士？

综合研究和混合方法。 EMNLP 2024 行业轨道。

[27] 李宽等. LaRA：对 检索增强生成（Retrieval-augmented Generation） 和长上下文法学硕士进行基准测试 - LC 或 RAG 路由（Routing） 没有灵丹妙药。 ICML 2025。

[28] 邹伟，耿润鹏，王丙辉，贾金元。 PoisonedRAG：对大语言 模型（Models） 的 检索增强生成（ Retrieval-augmented Generation） 的知识损坏攻击。 arXiv：2402.07867，2024。

[29] 迅良等. SafeRAG：检索（Retrieval） 中的安全基准测试 - 大语言模型（Large Language Model） 的增强版 生成（Generation）。访问控制2025。

[30] 沙尚克·什里德哈·巴特等人。企业人工智能必须强制执行参与者意识 访问控制（Access Control）。 arXiv：2509.14608，2025。

[31] 瓦伦丁·J·J·克莱德、约翰内斯·雷辛格、安德烈亚斯·费舍尔。

评估 检索增强生成（Retrieval-augmented Generation） 在学术文本上的 分块（Chunking） 策略。 arXiv：2607.01852，2026。

[32] 陈玉林，李浩然，隋媛，何宇飞，刘悦，宋阳秋，Bryan Hooi。间接 提示词注入（Prompt Injection） 攻击能否被检测和消除？访问控制2025。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

Depth checks

Area	What strong understanding looks like
Chunking	Explains the evidence-granularity trade-off and proposes corpus-specific evaluation instead of naming a magic token size.
Context packing	Treats top-k, deduplication, ordering, and compression as a token-budget optimization problem, not independent toggles.
Reliability	Separates retrieval failure from generation failure and understands that correct context can still yield an unsupported answer.
Attribution	Distinguishes answer correctness, support, citation coverage, and genuine reliance on evidence.
Security	Enforces authorization before context assembly and treats retrieved documents as untrusted input, not instructions.
Evaluation	Uses layer-specific metrics and representative slices, then validates automated judges against human-labeled production examples.

Additional concepts worth knowing

Prompt caching and context reuse. Repeated large context can sometimes be cached to reduce cost or latency, but cache identity and authorization become part of correctness.
Just-in-time agentic search. Agents can keep lightweight references and load information only when needed rather than preloading all potentially relevant context; this belongs to the broader context-engineering problem.
Structured and multimodal RAG. SQL, knowledge graphs, tables, images, and code often need retrieval or tool interfaces that preserve structure instead of flattening everything into text chunks.
Abstention and answerability. A production system should distinguish 'no relevant evidence found' from 'evidence found but insufficient/conflicting' and test whether the model declines appropriately.

References

[1] Patrick Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
[2] Anthropic. Effective context engineering for AI agents. Engineering blog, 2025.
[3] Yulong Hui, Yao Lu, Huanchen Zhang. UDA: A Benchmark Suite for Retrieval Augmented Generation in Real-world Document Analysis. NeurIPS 2024 Datasets and Benchmarks Track.
[4] Zhitong Wang et al. Document Segmentation Matters for Retrieval-Augmented Generation. Findings of ACL 2025.
[5] Anthropic. Introducing Contextual Retrieval. Engineering blog, 2024.
[6] Michael Gunther, Isabelle Mohr, Daniel James Williams, Bo Wang, Han Xiao. Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models. arXiv:2409.04701, 2024.
[7] Parth Sarthi et al. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. ICLR 2024.
[8] Nelson F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. TACL 2023.
[9] Liang Wang, Nan Yang, Furu Wei. Query2doc: Query Expansion with Large Language Models. EMNLP 2023.
[10] Luyu Gao, Xueguang Ma, Jimmy Lin, Jamie Callan. Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE). ACL 2023.
[11] Gordon V. Cormack, Charles L. A. Clarke, Stefan Buettcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009.
[12] Harsh Trivedi et al. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. ACL 2023.
[13] Zhengbao Jiang et al. Active Retrieval Augmented Generation (FLARE). EMNLP 2023.
[14] Akari Asai et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR 2024.
[15] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, Zhen-Hua Ling. Corrective Retrieval Augmented Generation. arXiv:2401.15884, 2024.
[16] Fangyuan Xu, Weijia Shi, Eunsol Choi. RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation. ICLR 2024.
[17] Huiqiang Jiang et al. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. ACL 2024.
[18] Hung-Ting Chen, Michael Zhang, Eunsol Choi. Rich Knowledge Sources Bring Complex Knowledge Conflicts: Recalibrating Models to Reflect Conflicting Evidence. EMNLP 2022.
[19] Zhuoran Jin et al. Tug-of-War between Knowledge: Exploring and Resolving Knowledge Conflicts in Retrieval-Augmented Language Models. LREC-COLING 2024.
[20] Tianyu Gao, Howard Yen, Jiatong Yu, Danqi Chen. Enabling Large Language Models to Generate Text with Citations (ALCE). EMNLP 2023.
[21] Jonas Wallat, Maria Heuss, Maarten de Rijke, Avishek Anand. Correctness is not Faithfulness in RAG Attributions. ICTIR 2025.
[22] Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. EACL 2024 System Demonstrations.
[23] Jon Saad-Falcon, Omar Khattab, Christopher Potts, Matei Zaharia. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. NAACL 2024.
[24] Dongyu Ru et al. RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation. NeurIPS 2024 Datasets and Benchmarks Track.
[25] Darren Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130, revised 2025.
[26] Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, Michael Bendersky. Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach. EMNLP 2024 Industry Track.
[27] Kuan Li et al. LaRA: Benchmarking Retrieval-Augmented Generation and Long-Context LLMs - No Silver Bullet for LC or RAG Routing. ICML 2025.
[28] Wei Zou, Runpeng Geng, Binghui Wang, Jinyuan Jia. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. arXiv:2402.07867, 2024.
[29] Xun Liang et al. SafeRAG: Benchmarking Security in Retrieval-Augmented Generation of Large Language Model. ACL 2025.
[30] Shashank Shreedhar Bhatt et al. Enterprise AI Must Enforce Participant-Aware Access Control. arXiv:2509.14608, 2025.
[31] Valentin J. J. Kreileder, Johannes Reisinger, Andreas Fischer. Evaluating Chunking Strategies for Retrieval-Augmented Generation on Academic Texts. arXiv:2607.01852, 2026.
[32] Yulin Chen, Haoran Li, Yuan Sui, Yufei He, Yue Liu, Yangqiu Song, Bryan Hooi. Can Indirect Prompt Injection Attacks Be Detected and Removed? ACL 2025.